

Aim: Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

Code A:1:print("~~~~~Hariprasad Vishwakarma T127~~~~~")

print("Caesar Cipher")

def encrypt(text, shift):

result = ""

for char in text:

if char.isalpha():

if char.isupper():

result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))

else:

result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))

else:

result += char

return result

def decrypt(cipher_text, shift):

result = ""

for char in cipher_text:

if char.isalpha():

if char.isupper():

result += chr((ord(char) - ord('A') - shift) % 26 + ord('A'))

else:

result += chr((ord(char) - ord('a') - shift) % 26 + ord('a'))

else:

result += char

return result

Main Program

text = input("Enter the message: ")

shift = int(input("Enter the shift value: "))

encrypted = encrypt(text, shift)

print("Encrypted Message:", encrypted)

decrypted = decrypt(encrypted, shift)

print("Decrypted Message:", decrypted)

Hariprasad Vishwakarma

T127

TYCS

MVLU COLLEGE

Output:



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: D:/T127/cyber security1A.py =====
~~~~~Hariprasad Vishwakarma T127~~~~~
Caesar Cipher
Enter the message: QWERTY
Enter the shift value: 852
Encrypted Message: KQYLNS
Decrypted Message: QWERTY
>>>
```

```
Code A :2:import tkinter as tk
from tkinter import messagebox
```

```
# ----- Caesar Cipher Functions ----- #
```

```
def caesar_encrypt(text, shift):
    result = ""

    for ch in text:
        if ch.isalpha():
            if ch.isupper():
                result += chr((ord(ch) - ord('A') + shift) % 26 + ord('A'))
            else:
                result += chr((ord(ch) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += ch

    return result
```

```
def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)
```

Hariprasad Vishwakarma
T127
TYCS

```
# ----- Button Functions ----- #
```

```
def encrypt_text():
```

```
    try:
```

```
        text = txt_message.get()
```

```
        shift = int(txt_shift.get())
```

```
        encrypted = caesar_encrypt(text, shift)
```

```
        result_box.config(state="normal")
```

```
        result_box.delete(0, tk.END)
```

```
        result_box.insert(0, encrypted)
```

```
        result_box.config(state="readonly")
```

```
    except ValueError:
```

```
        messagebox.showerror("Error", "Enter a valid shift value.")
```

```
def decrypt_text():
```

```
    try:
```

```
        text = txt_message.get()
```

```
        shift = int(txt_shift.get())
```

```
        decrypted = caesar_decrypt(text, shift)
```

```
        result_box.config(state="normal")
```

```
        result_box.delete(0, tk.END)
```

```
        result_box.insert(0, decrypted)
```

```
        result_box.config(state="readonly")
```

```
    except ValueError:
```

```
        messagebox.showerror("Error", "Enter a valid shift value.")
```

```
def clear_all():
```

```
    txt_message.delete(0, tk.END)
```

```
    txt_shift.delete(0, tk.END)
```

```
    result_box.config(state="normal")
```

```
    result_box.delete(0, tk.END)
```

```
    result_box.config(state="readonly")
```

```
# ----- GUI ----- #
```

Hariprasad Vishwakarma

T127

TYCS

```

root = tk.Tk()
root.title("Classical Substitution Cipher - Caesar Cipher")
root.geometry("500x320")
root.resizable(False, False)

title = tk.Label(
    root,
    text="Classical Substitution Cipher (Caesar Cipher)",
    font=("Arial", 15, "bold")
)
title.pack(pady=15)

# Message
tk.Label(root, text="Enter Message:", font=("Arial", 11)).pack()

txt_message = tk.Entry(root, width=50, font=("Arial", 11))
txt_message.pack(pady=5)

# Shift
tk.Label(root, text="Shift Value:", font=("Arial", 11)).pack()

txt_shift = tk.Entry(root, width=10, font=("Arial", 11))
txt_shift.pack(pady=5)

# Buttons
frame = tk.Frame(root)
frame.pack(pady=15)

btn_encrypt = tk.Button(
    frame,
    text="Encrypt",
    width=12,
    bg="lightgreen",
    command=encrypt_text
)
btn_encrypt.grid(row=0, column=0, padx=10)

btn_decrypt = tk.Button(
    frame,
    text="Decrypt",
    width=12,
    bg="lightblue",
    command=decrypt_text
)

```

Hariprasad Vishwakarma
T127
TYCS

```

)
btn_decrypt.grid(row=0, column=1, padx=10)

btn_clear = tk.Button(
    frame,
    text="Clear",
    width=12,
    bg="tomato",
    command=clear_all
)
btn_clear.grid(row=0, column=2, padx=10)

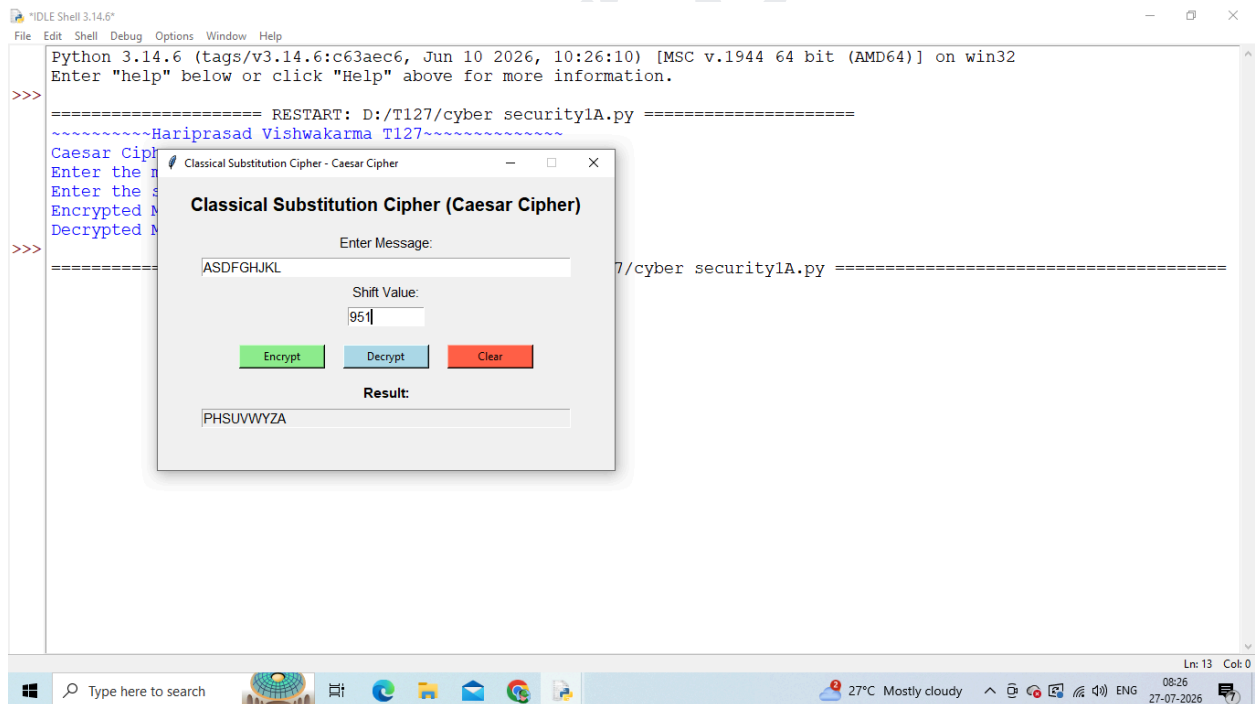
# Result
tk.Label(root, text="Result:", font=("Arial", 11, "bold")).pack()

result_box = tk.Entry(root, width=50, font=("Arial", 11), state="readonly")
result_box.pack(pady=5)

root.mainloop()

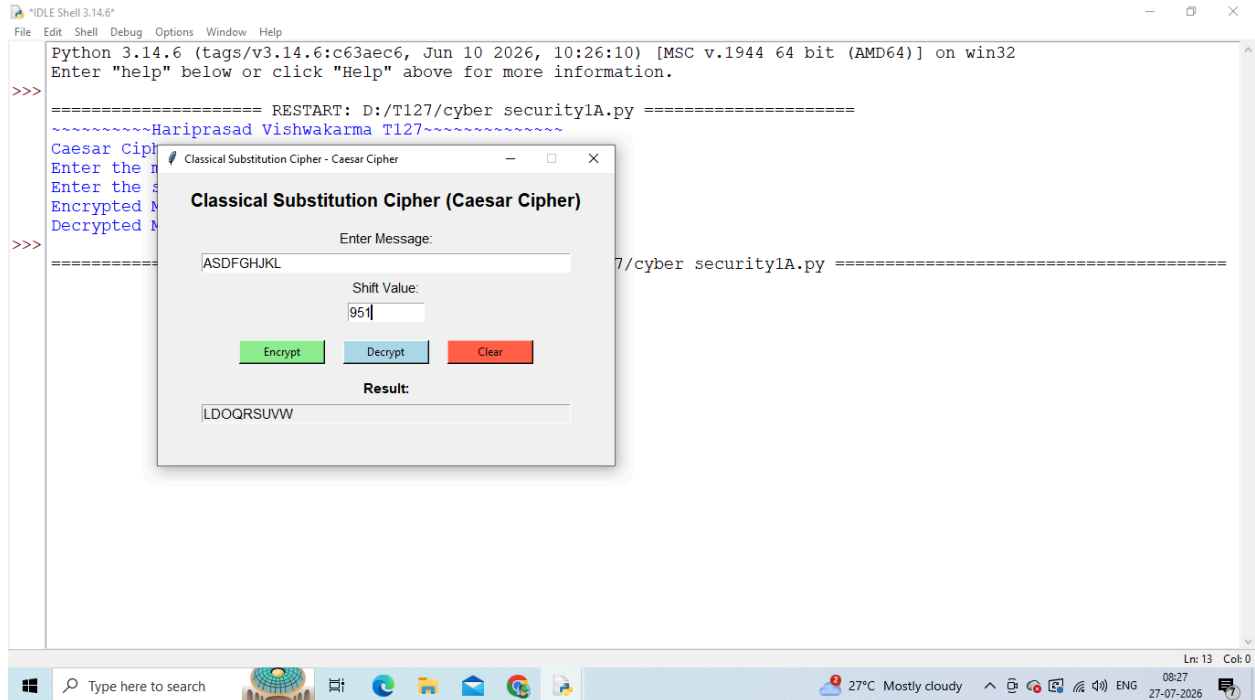
```

Output:



Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE



Code B:1:### Rail Fence Cipher (Transposition Technique)

```
print("Hariprasad Vishwakarma T127")
```

```
print("Rail Fence Cipher (Transposition Technique)")
```

```
print()
```

```
# Encryption Function
```

```
def encrypt(text, key):
```

```
    rail = [['\n' for i in range(len(text))] for j in range(key)]
```

```
    direction_down = False
```

```
    row, col = 0, 0
```

```
    for char in text:
```

```
        if row == 0 or row == key - 1:
```

```
            direction_down = not direction_down
```

```
        rail[row][col] = char
```

```
        col += 1
```

```
    if direction_down:
```

```
        row += 1
```

```
    else:
```

```
        row -= 1
```

```
    result = ""
```

```
    for i in range(key):
```

Hariprasad Vishwakarma

T127

TYCS

```

for j in range(len(text)):
    if rail[i][j] != '\n':
        result += rail[i][j]

```

```

return result

```

```

# Decryption Function

```

```

def decrypt(cipher, key):
    rail = [['\n' for i in range(len(cipher))] for j in range(key)]

```

```

    direction_down = None
    row, col = 0, 0

```

```

    for i in range(len(cipher)):
        if row == 0:
            direction_down = True
        if row == key - 1:
            direction_down = False

```

```

        rail[row][col] = '*'
        col += 1

```

```

        if direction_down:
            row += 1
        else:
            row -= 1

```

```

    index = 0
    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

```

```

    result = ""
    row, col = 0, 0

```

```

    for i in range(len(cipher)):
        if row == 0:
            direction_down = True
        if row == key - 1:
            direction_down = False

```

Hariprasad Vishwakarma

T127

TYCS

```
result += rail[row][col]
col += 1
```

```
if direction_down:
    row += 1
else:
    row -= 1
```

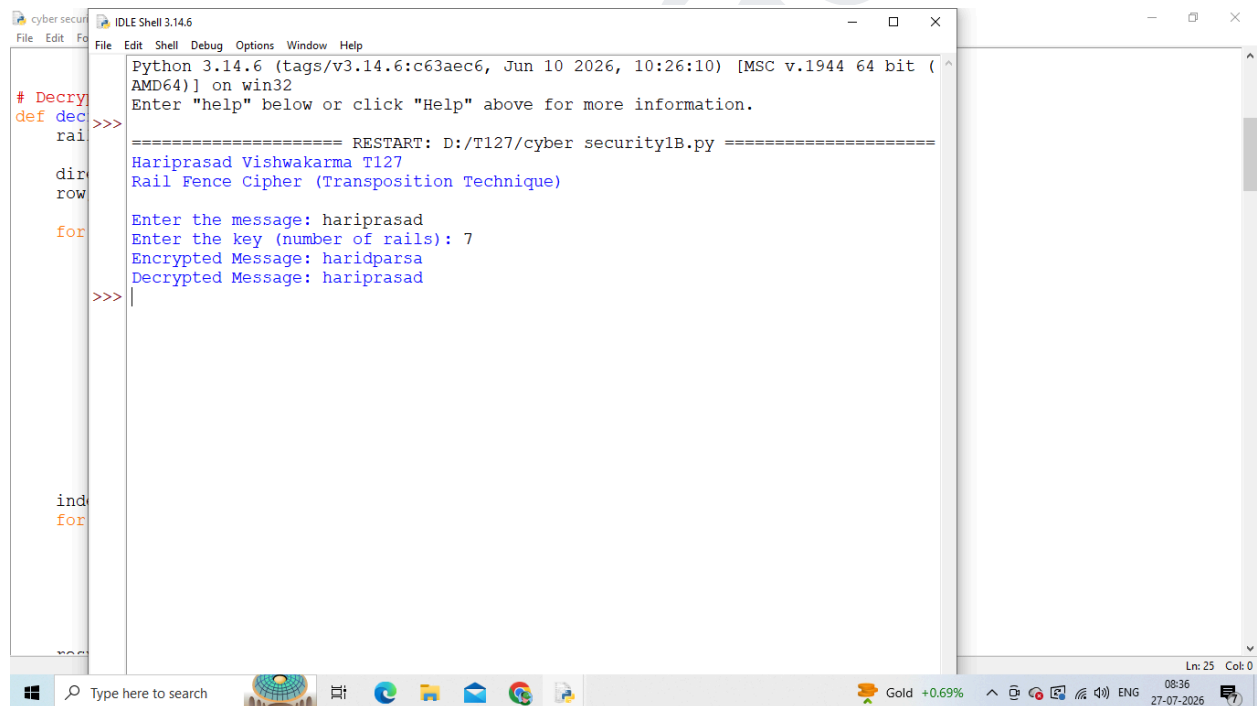
```
return result
```

```
text = input("Enter the message: ")
key = int(input("Enter the key (number of rails): "))
```

```
encrypted = encrypt(text, key)
print("Encrypted Message:", encrypted)
```

```
decrypted = decrypt(encrypted, key)
print("Decrypted Message:", decrypted)
```

Output:



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: D:/T127/cyber security1B.py =====
Hariprasad Vishwakarma T127
Rail Fence Cipher (Transposition Technique)

Enter the message: hariprasad
Enter the key (number of rails): 7
Encrypted Message: haridparsa
Decrypted Message: hariprasad

>>>
```


Code B:2:

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
# ----- Encryption ----- #
```

```
def encrypt(text, key):
```

```
    if key <= 1:
```

```
        return text
```

```
    rail = [['\n' for _ in range(len(text))] for _ in range(key)]
```

```
    row = 0
```

```
    col = 0
```

```
    direction_down = False
```

```
    for ch in text:
```

```
        if row == 0 or row == key - 1:
```

```
            direction_down = not direction_down
```

```
        rail[row][col] = ch
```

```
        col += 1
```

```
        if direction_down:
```

```
            row += 1
```

```
        else:
```

```
            row -= 1
```

```
    result = ""
```

```
    for i in range(key):
```

```
        for j in range(len(text)):
```

```
            if rail[i][j] != '\n':
```

```
                result += rail[i][j]
```

```
    return result
```

```
# ----- Decryption ----- #
```

```
def decrypt(cipher, key):
```

```
    if key <= 1:
```

Hariprasad Vishwakarma

T127

TYCS

```

return cipher

rail = [['\n' for _ in range(len(cipher))] for _ in range(key)]

row = 0
col = 0
direction_down = None

# Mark zig-zag path
for i in range(len(cipher)):

    if row == 0:
        direction_down = True
    if row == key - 1:
        direction_down = False

    rail[row][col] = '*'
    col += 1

    if direction_down:
        row += 1
    else:
        row -= 1

# Fill marked places
index = 0
for i in range(key):
    for j in range(len(cipher)):
        if rail[i][j] == '*' and index < len(cipher):
            rail[i][j] = cipher[index]
            index += 1

result = ""
row = 0
col = 0

for i in range(len(cipher)):

    if row == 0:
        direction_down = True
    if row == key - 1:
        direction_down = False

    result += rail[row][col]

```

Hariprasad Vishwakarma

T127

TYCS

```
col += 1

if direction_down:
    row += 1
else:
    row -= 1

return result

def encrypt_text():
    try:
        text = message_entry.get()
        key = int(key_entry.get())

        if key < 2:
            messagebox.showerror("Error", "Key must be greater than 1.")
            return

        result = encrypt(text, key)

        result_entry.config(state="normal")
        result_entry.delete(0, tk.END)
        result_entry.insert(0, result)
        result_entry.config(state="readonly")

    except ValueError:
        messagebox.showerror("Error", "Enter a valid key.")

def decrypt_text():
    try:
        text = message_entry.get()
        key = int(key_entry.get())

        if key < 2:
            messagebox.showerror("Error", "Key must be greater than 1.")
            return

        result = decrypt(text, key)

        result_entry.config(state="normal")
        result_entry.delete(0, tk.END)
        result_entry.insert(0, result)
        result_entry.config(state="readonly")
```

Hariprasad Vishwakarma

T127

TYCS

```
except ValueError:
    messagebox.showerror("Error", "Enter a valid key.")

def clear():
    message_entry.delete(0, tk.END)
    key_entry.delete(0, tk.END)

    result_entry.config(state="normal")
    result_entry.delete(0, tk.END)
    result_entry.config(state="readonly")

root = tk.Tk()
root.title("Rail Fence Cipher - Transposition Technique")
root.geometry("550x330")
root.resizable(False, False)

title = tk.Label(
    root,
    text="Rail Fence Cipher (Transposition Technique)",
    font=("Arial", 16, "bold")
)
title.pack(pady=15)

tk.Label(root, text="Enter Message:", font=("Arial", 11)).pack()

message_entry = tk.Entry(root, width=50, font=("Arial", 11))
message_entry.pack(pady=5)

tk.Label(root, text="Enter Key (Rails):", font=("Arial", 11)).pack()

key_entry = tk.Entry(root, width=10, font=("Arial", 11))
key_entry.pack(pady=5)

frame = tk.Frame(root)
frame.pack(pady=15)

encrypt_btn = tk.Button(
    frame,
    text="Encrypt",
    width=12,

```

Hariprasad Vishwakarma
T127
TYCS

```

        bg="lightgreen",
        command=encrypt_text
    )
    encrypt_btn.grid(row=0, column=0, padx=10)

    decrypt_btn = tk.Button(
        frame,
        text="Decrypt",
        width=12,
        bg="lightblue",
        command=decrypt_text
    )
    decrypt_btn.grid(row=0, column=1, padx=10)

    clear_btn = tk.Button(
        frame,
        text="Clear",
        width=12,
        bg="tomato",
        command=clear
    )
    clear_btn.grid(row=0, column=2, padx=10)

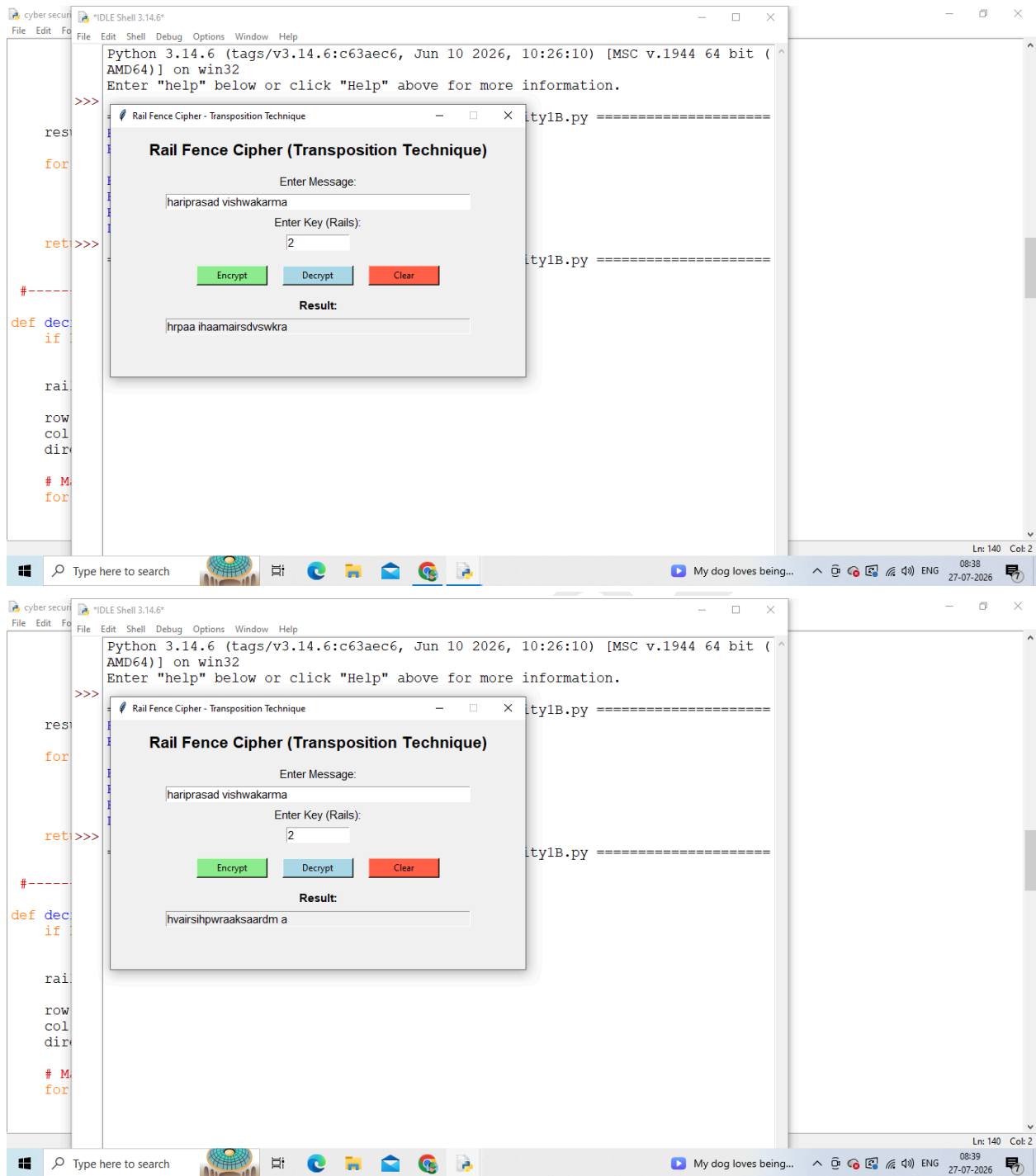
    tk.Label(root, text="Result:", font=("Arial", 11, "bold")).pack()

    result_entry = tk.Entry(root, width=50, font=("Arial", 11), state="readonly")
    result_entry.pack(pady=5)

    root.mainloop()

```

Output:



Hariprasad Vishwakarma
T127
TYCS